gc_content.py 사용법 및 코드 설명서

대상 파일: lab11/src/gc_content.py

1. 프로그램의 목적

FASTQ 파일에 들어 있는 **모든 read**의 GC 함량과 평균 Phred 품질 점수를 계산한 뒤, 다음 두 가지를 출력한다.

1. **최고·최저 read 1개씩**: GC 함량이 가장 높은 read와 가장 낮은 read의 이름, GC 함량(%), 평균 품질
2. **상위 10개·하위 10개 그룹**: GC 함량이 높은 순서로 10개, 낮은 순서로 10개 read를 골라 각각 표로 출력하고, **그룹별 평균 품질**을 구한다

두 경우 모두 **전체 read의 평균 품질**과 비교하여 차이(값 - 전체 평균)를 함께 출력한다. read 1개만 비교하면 우연한 결과일 수 있으므로, 10개씩 묶은 그룹으로도 비교하는 것이 이번 버전의 목적이다.

2. 실행 방법

lab11 폴더에서 다음과 같이 실행한다.

```
python3 src/gc_content.py data/SRR1553607_1.fastq
```

형식:

```
python3 src/gc_content.py <FASTQ 파일>
```

- 인자는 sys.argv[1]로 받는 FASTQ 파일 경로 하나이다.
- 인자 개수를 검사하는 코드는 없다. 인자를 주지 않으면 IndexError가 난다.

3. 입력 형식

FASTQ 형식의 텍스트 파일이다. read 하나는 **4줄**로 이루어진다.

줄	내용	예 (data/SRR1553607_1.fastq의 첫 read)
1	@로 시작하는 read 이름	@SRR1553607.1 1 length=101
2	염기 서열	GTTAGCGTTGTTGATCGCG...
3	+ 줄	+SRR1553607.1 1 length=101
4	품질 문자열 (Phred+33, 서열과 길이가 같음)	BB@FFFFFHHHHHJJJJJ...

- 파일이 정확히 4줄 단위라고 가정한다.
- 서열은 대문자라고 가정한다. 소문자 g, c는 GC로 세지 않는다.
- 길이가 0인 read가 있으면 나눗셈에서 ZeroDivisionError가 난다 (이 파일의 read는 모두 101 bp이므로 해당 없음).
- read가 10개보다 적으면 상위·하위 그룹에는 있는 read만큼만 들어간다 ([0:10] 슬라이싱은 오류 없이 있는 만큼만 잘라 준다).

4. 출력 형식

화면(표준 출력)에 출력하며 파일로 저장하지 않는다. 숫자는 round(값, 2)로 소수점 아래 둘째 자리까지 반올림한다. 출력은 6개 부분으로 나뉜다.

```
분석한 read 개수: <read 수>

[GC 함량이 가장 높은 read]
이름: <read 이름>
GC 함량(%): <숫자>
평균 품질: <숫자>

[GC 함량이 가장 낮은 read]
이름: <read 이름>
GC 함량(%): <숫자>
평균 품질: <숫자>

[품질 비교]
전체 read 평균 품질: <숫자>
GC 최고 read - 전체 평균: <숫자>
GC 최저 read - 전체 평균: <숫자>

[GC 함량 상위 10개 read]
순위    이름    GC 함량(%)    평균 품질
1    <read 이름>    <숫자>    <숫자>
...  (10번째 줄까지)

[GC 함량 하위 10개 read]
순위    이름    GC 함량(%)    평균 품질
1    <read 이름>    <숫자>    <숫자>
...  (10번째 줄까지)

[그룹별 품질 비교]
상위 10개 그룹 평균 품질: <숫자>
하위 10개 그룹 평균 품질: <숫자>
전체 read 평균 품질: <숫자>
상위 10개 그룹 - 전체 평균: <숫자>
하위 10개 그룹 - 전체 평균: <숫자>
```

- read 이름은 @를 뺀 1번째 줄 전체이다.
- 상위·하위 10개 표는 **탭(₩t)으로 구분**된다. 상위 표의 1위는 GC 함량이 가장 높은 read, 하위 표의 1위는 GC 함량이 가장 낮은 read이다.
- ... - 전체 평균 줄이 음수이면 전체 평균보다 품질이 낮다는 뜻이다.

5. 계산 공식

5.1 GC 함량

```
GC 함량(%) = (G 개수 + C 개수) / 서열 길이 × 100
```

예: 101 bp read에 G와 C가 합쳐 99개이면 99 / 101 × 100 = 98.02%.

5.2 read 하나의 평균 품질 (Phred+33)

품질 문자 하나의 점수는 ord(문자) - 33이다. 예: 'J' → 74 - 33 = 41, '#' → 35 - 33 = 2.

```
read 평균 품질 = (모든 품질 문자의 점수 합) / 품질 문자열 길이
```

5.3 전체 read 평균 품질

qual_summary.py와 같은 방식으로, **read마다 평균 품질을 먼저 구한 뒤 그 값들을 다시 평균** 낸다.

```
전체 평균 품질 = (read별 평균 품질의 합) / read 개수
```

5.4 그룹 평균 품질

전체 평균과 같은 방식으로, 그룹에 들어 있는 read들의 **read별 평균 품질을 다시 평균** 낸다.

```
그룹 평균 품질 = (그룹에 속한 read별 평균 품질의 합) / 그룹의 read 개수(보통 10)
```

예 (8장의 실제 상위 10개):

```
(11.10 + 19.73 + 9.06 + 9.25 + 24.01 + 32.28 + 17.33 + 27.47 + 5.96 + 31.53) / 10
= 187.72 / 10 = 18.772 ≃ 18.77
```

(위 덧셈은 표에 보이는 반올림 값을 쓴 것이고, 프로그램은 반올림하기 전의 값으로 계산한다. 이 경우 결과는 같다.)

6. 동점(같은 GC 함량) 처리 기준

GC 함량이 같은 read가 여러 개 있으면, 파일에서 먼저 나온 read를 우선한다. 최고·최저 read와 상위·하위 10개 모두 같은 기준이지만 작동 원리는 다르다.

6.1 최고·최저 read 1개

비교할 때 >=, <=가 아니라 >와 <를 쓰기 때문이다. 뒤에 나온 read의 GC 함량이 지금까지의 최고값과 **같으면** gc > max_gc가 거짓이 되어 저장된 read가 바뀌지 않는다. 최저값도 마찬가지이다.

6.2 상위·하위 10개

Python의 sorted()는 **안정 정렬(stable sort)** 이다. 정렬 기준값(GC 함량)이 같은 항목끼리는 정렬 전의 순서를 그대로 유지한다. results 리스트는 파일 순서대로 만들어졌으므로, 동점인 read는 파일에서 먼저 나온 것이 앞에 온다. reverse=True로 큰 값부터 정렬할 때도 같은 값끼리의 순서는 뒤집히지 않는다.

그래서 10위 경계에 동점 read가 많으면 **파일에서 먼저 나온 read만 10개 안에 들어가고, 나머지는 잘린다.** 실제 데이터에서 이런 경우가 생겼다 (8.3절 참고).

7. 주요 코드의 작동 원리

7.1 read_fastq(filename) — FASTQ 파일 읽기

Lab 11의 parse_fastq.py, qual_summary.py에서 쓴 함수와 같다.

```
def read_fastq(filename):
    records = []
    with open(filename) as f:
        lines = f.readlines()
    for i in range(0, len(lines), 4):
        name = lines[i].rstrip()[1:]
        seq = lines[i + 1].rstrip()
        qual = lines[i + 3].rstrip()
        records.append((name, seq, qual))
    return records
```

- 파일 전체를 줄 리스트로 읽고, i를 0, 4, 8, ...으로 4씩 늘리며 read 하나씩 처리한다.
- 1번째 줄에서 @를 뺀 것이 이름, 2번째 줄이 서열, 4번째 줄이 품질 문자열이다.
- (이름, 서열, 품질) 튜플의 리스트를 돌려준다.

7.2 gc_content(seq) — GC 함량 계산

```
def gc_content(seq):
    gc = 0
    for base in seq:
        if base == "G" or base == "C":
            gc = gc + 1
    return gc / len(seq) * 100
```

- 서열을 한 글자씩 보면서 G나 C이면 gc를 1 늘린다.
- 다 센 뒤 서열 길이로 나누고 100을 곱해 백분율로 돌려준다.

7.3 mean_quality(qual) — read 하나의 평균 품질

qual_summary.py의 함수와 같다.

```
def mean_quality(qual):
    scores = []
    for ch in qual:
        scores.append(ord(ch) - 33)
    return sum(scores) / len(scores)
```

- 품질 문자마다 ord(ch) - 33으로 Phred 점수를 구해 리스트에 모은 뒤 평균을 낸다.

7.4 모든 read 계산 + 최고·최저 read 찾기 (한 번의 for문)

```
max_gc = -1
min_gc = 101
total_q = 0
results = []

for name, seq, qual in reads:
    gc = gc_content(seq)
    q = mean_quality(qual)
    total_q = total_q + q
    results.append((name, gc, q))

    if gc > max_gc:
        max_name = name
        max_gc = gc
        max_q = q
```

```
    if gc < min_gc:
        min_name = name
        min_gc = gc
        min_q = q

all_q = total_q / len(reads)
```

1. 시작값: max_gc는 어떤 GC 함량(0 이상)보다 작은 -1, min_gc는 어떤 GC 함량(100 이하)보다 큰 101로 둔다. 그래서 첫 read는 반드시 최고값과 최저값 양쪽에 저장된다.
2. read마다 GC 함량 gc와 평균 품질 q를 구하고, q를 total_q에 더한다.
3. (이름, GC 함량, 평균 품질) 튜플을 results 리스트에 **파일 순서대로** 추가한다. 이 리스트가 7.5절의 정렬 재료가 된다.
4. gc가 지금까지의 최고값보다 **크면** 이름, GC 함량, 평균 품질을 새로 저장한다. 최저값도 같은 방식으로 **작을 때만** 바꾼다 (6.1절).
5. 반복이 끝나면 total_q를 read 개수로 나누어 전체 평균 품질 all_q를 구한다.

7.5 상위 10개·하위 10개 고르기 — get_gc()와 sorted()

```
def get_gc(record):
    return record[1]

high_sorted = sorted(results, key=get_gc, reverse=True)   # GC 높은 순
low_sorted = sorted(results, key=get_gc)                  # GC 낮은 순
top10 = high_sorted[0:10]
bottom10 = low_sorted[0:10]
```

- get_gc()는 튜플 (이름, GC 함량, 평균 품질)에서 두 번째 값(인덱스 1)인 GC 함량을 돌려준다. sorted()의 key=에 이 함수를 넘기면 **GC 함량을 기준으로** 정렬한다.
- reverse=True를 주면 큰 값부터(내림차순), 주지 않으면 작은 값부터(오름차순) 정렬한다.
- 정렬된 리스트에서 앞의 10개([0:10], 인덱스 0~9)를 잘라 각각 상위 10개, 하위 10개로 쓴다.
- sorted()는 원래 리스트 results를 바꾸지 않고 새 리스트를 만든다. 그래서 같은 results로 두 번 정렬할 수 있다.
- 동점 처리는 6.2절과 같다.

7.6 print_group(title, group) — 그룹 표 출력과 그룹 평균 품질

```
def print_group(title, group):
    print(title)
    print("순위\t이름\tGC 함량(%)\t평균 품질")
    group_total = 0
    rank = 1
    for name, gc, q in group:
        print(str(rank) + "\t" + name + "\t" + str(round(gc, 2)) + "\t" + str(round(q, 2)))
        group_total = group_total + q
        rank = rank + 1
    return group_total / len(group)
```

이 함수는 **두 가지 일**을 한 번에 한다.

1. **출력**: 제목과 머리글 줄을 출력한 뒤, 그룹의 read를 하나씩 순위, 이름, GC 함량, 평균 품질 순서로 탭으로 이어 붙여 출력한다. rank는 1부터 시작해 한 줄마다 1씩 늘어난다.
2. **그룹 평균 계산**: 같은 반복문 안에서 read별 평균 품질 q를 group_total에 더해 두었다가, 마지막에 len(group)(보통 10)으로 나누어 **돌려준다(return)**.

호출하는 쪽에서는 돌려받은 값을 변수에 저장해 둔다.

```
top_q = print_group("[GC 함량 상위 10개 read]", top10)
bottom_q = print_group("[GC 함량 하위 10개 read]", bottom10)
```

7.7 그룹별 품질 비교 출력

```
print("상위 10개 그룹 평균 품질: " + str(round(top_q, 2)))
print("하위 10개 그룹 평균 품질: " + str(round(bottom_q, 2)))
print("전체 read 평균 품질: " + str(round(all_q, 2)))
print("상위 10개 그룹 - 전체 평균: " + str(round(top_q - all_q, 2)))
print("하위 10개 그룹 - 전체 평균: " + str(round(bottom_q - all_q, 2)))
```

- 차이는 반올림하기 전의 값끼리 뺀 뒤 반올림한다. 그래서 화면에 보이는 반올림 값끼리 빼면 끝자리가 0.01 다를 수 있다.

8. 테스트 결과

8.1 실행 결과

lab11 폴더에서 data/SRR1553607_1.fastq로 실제 실행한 결과이다 (실행 시간 약 2~7초).

```
$ python3 src/gc_content.py data/SRR1553607_1.fastq
분석한 read 개수: 203445

[GC 함량이 가장 높은 read]
이름: SRR1553607.53820 53820 length=101
GC 함량(%): 98.02
평균 품질: 11.1

[GC 함량이 가장 낮은 read]
이름: SRR1553607.166164 166164 length=101
GC 함량(%): 12.87
평균 품질: 39.66

[품질 비교]
전체 read 평균 품질: 33.45
GC 최고 read - 전체 평균: -22.35
GC 최저 read - 전체 평균: 6.21

[GC 함량 상위 10개 read]
순위    이름    GC 함량(%)    평균 품질
1    SRR1553607.53820 53820 length=101    98.02    11.1
2    SRR1553607.169331 169331 length=101    95.05    19.73
3    SRR1553607.77585 77585 length=101    94.06    9.06
4    SRR1553607.143780 143780 length=101    94.06    9.25
5    SRR1553607.106561 106561 length=101    93.07    24.01
6    SRR1553607.169682 169682 length=101    93.07    32.28
7    SRR1553607.194620 194620 length=101    93.07    17.33
8    SRR1553607.15006 15006 length=101    92.08    27.47
9    SRR1553607.54659 54659 length=101    92.08    5.96
10    SRR1553607.173565 173565 length=101    92.08    31.53

[GC 함량 하위 10개 read]
순위    이름    GC 함량(%)    평균 품질
1    SRR1553607.166164 166164 length=101    12.87    39.66
```

```
2    SRR1553607.165570 165570 length=101    13.86    38.32
3    SRR1553607.179986 179986 length=101    13.86    38.09
4    SRR1553607.59968 59968 length=101    15.84    39.58
5    SRR1553607.61985 61985 length=101    15.84    33.28
6    SRR1553607.70984 70984 length=101    15.84    39.22
7    SRR1553607.117728 117728 length=101    15.84    39.27
8    SRR1553607.117814 117814 length=101    15.84    37.28
9    SRR1553607.166721 166721 length=101    15.84    40.11
10    SRR1553607.172662 172662 length=101    15.84    38.46

[그룹별 품질 비교]
상위 10개 그룹 평균 품질: 18.77
하위 10개 그룹 평균 품질: 38.33
전체 read 평균 품질: 33.45
상위 10개 그룹 - 전체 평균: -14.68
하위 10개 그룹 - 전체 평균: 4.88
```

8.2 검증

Python 프로그램과 별도로 awk로 GC 개수와 평균 품질을 다시 계산하여 비교하였다.

확인 항목	방법	결과
read 개수	서열 줄 개수 세기	203445 — 일치 (모든 read 101 bp)
전체 평균 품질	기존 qual_summary.py 실행	33.4497222958299 → 반올림 33.45 — 일치
최고/최저 GC read	awk로 read별 GC 개수 계산	GC 99개(98.02%) 1개, GC 13개(12.87%) 1개 — 같은 read, 일치
상위 10개 구성	awk로 GC 개수별 read 수 세기	99개 1, 96개 1, 95개 2, 94개 3 → 7개, 나머지 3자리는 93개(92.08%) read 6개 중 파일 앞쪽 3개 — 일치
하위 10개 구성	같은 방법	13개 1, 14개 2, 16개(15.84%) 7 → 정확히 10개 (15개인 read는 없음) — 일치
read별 평균 품질	awk로 품질 문자를 점수로 바꿔 다시 계산	20개 read 모두 표의 값과 일치
그룹 평균 품질	awk로 구한 read별 값 10개씩 평균	상위 18.771, 하위 38.326 → 18.77 / 38.33 — 일치

8.3 동점 처리 확인 (10위 경계)

상위 10개: GC 함량 92.08%(GC 93개)인 read가 파일 전체에 **6개** 있지만 남은 자리는 3개(8~10위)뿐이다. 6.2절의 기준대로 파일에서 먼저 나온 3개만 들어갔다.

GC 92.08% read	평균 품질	상위 10개 포함 여부
SRR1553607.15006	27.47	포함 (8위)
SRR1553607.54659	5.96	포함 (9위)
SRR1553607.173565	31.53	포함 (10위)
SRR1553607.176568	5.75	제외
SRR1553607.185657	10.21	제외
SRR1553607.195126	34.17	제외

참고로 제외된 3개를 대신 넣으면 상위 그룹 평균 품질은 약 17.29가 된다. 즉 **동점 처리 기준에 따라 그룹 평균이 약 1.5 달라질 수 있으므로**, 그룹 평균은 대략적인 경향으로만 보아야 한다.

하위 10개: GC 함량 15.84%(GC 16개)인 read가 정확히 7개이고 남은 자리도 7개(4~10위)이므로 잘린 read가

없다.

8.4 결과 해석

- **최고·최저 read 1개씩**: GC 최고 read(98.02%)는 평균 품질 11.1로 전체 평균(33.45)보다 22.35 낮고, 35번째 위치부터 끝까지 품질 문자가 모두 #(Phred 2)이다. 서열 뒷부분에 G가 길게 이어지는데, 시퀀싱 신호가 약해질 때 생기는 인공 서열일 가능성이 있다. GC 최저 read(12.87%)는 평균 품질 39.66으로 대부분의 위치가 J(Phred 41)인 좋은 read이다.
- **상위 10개 그룹**: 평균 품질 18.77로 전체 평균보다 14.68 낮다. 10개 모두 전체 평균(33.45)보다 낮고 (가장 높은 값도 32.28), 5.96, 9.06, 9.25처럼 매우 낮은 read도 있다. 따라서 GC 함량이 극단적으로 높은 read는 **품질이 낮은 read가 많다**고 볼 수 있다.
- **하위 10개 그룹**: 평균 품질 38.33으로 전체 평균보다 4.88 높다. 10개 모두 33 이상이며, 가장 낮은 33.28도 전체 평균과 거의 같다.
- **read 1개 비교와의 차이**: 최고 read 1개의 차이는 -22.35였지만, 10개 그룹의 차이는 -14.68로 줄었다. read 1개만 보면 차이가 과장될 수 있으며, 그룹으로 보아도 같은 방향의 차이가 유지된다는 점이 확인되었다.
- **주의**: 양 끝의 20개 read만 본 결과이므로 "GC 함량과 품질이 반비례한다"고 일반화할 수는 없다. 이 프로그램은 전체 read에 대한 상관관계를 계산하지 않는다. 또 8.3절처럼 동점 처리에 따라 그룹 구성이 달라질 수 있다.